SISTEMAS DISTRIBUIDOS

LABORATORIO S09

Que el aviso no tumbe la transferencia

Fecha de entrega: 23 de octubre, 2026
Profesor: Ing. Oscar Manual Ruiz Hurtado

Alumno: Echeverría Goicochea Erick Isaac
Equipo: DiDistribuidores

No. Cuenta: 424153520
Grupo: 2
Semestre: 2027-1

Bitácora Laboratorio S09 · Mexi Banco: que el aviso no tumbe la transferencia

Sistemas Distribuidos 2027-1 · Grupo 2 · S09

Alumno: Echeverria Goicochea Erick Isaac

1. El problema (Parte 0) [21/10/26 – 12:02 PM]

Qué hice. Levanté el banco con docker compose y abrí el panel de RabbitMQ. Antes de tocar código verifiqué que no existía ningún exchange propio ni colas: solo los exchanges que trae RabbitMQ. Después creé las cuentas de Ana (1000) y Beto (500) por el gateway (Ilustración 3). Por último apagué notificación con docker compose stop notificacion, transferí 50 de Ana a Beto, consulté el estado y los saldos, y volví a encender notificación (Ilustración 4).

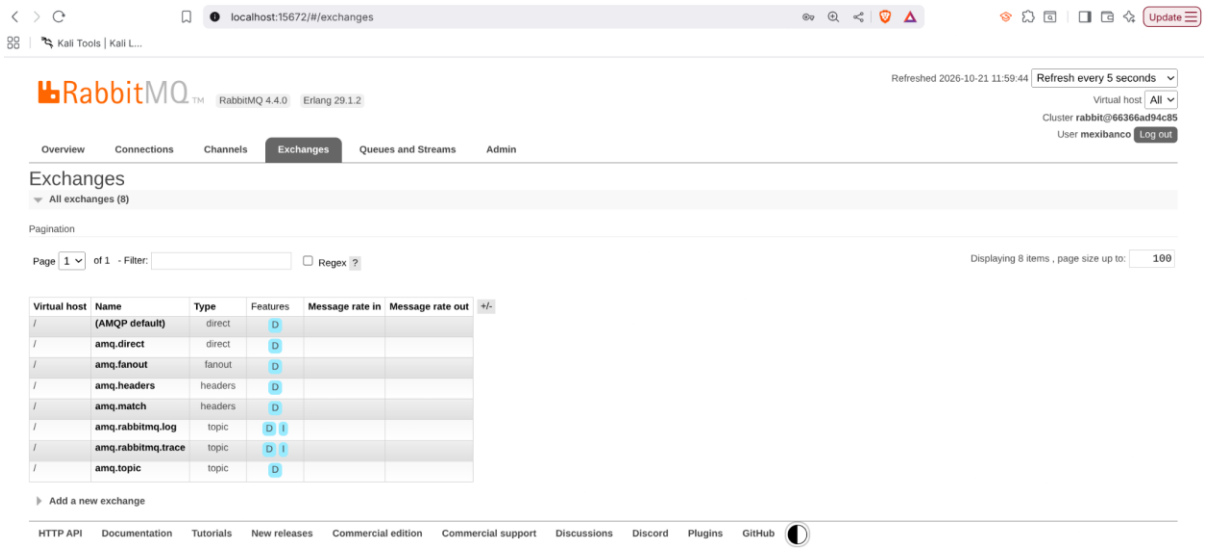


Ilustración 1. Panel de RabbitMQ, pestaña Exchanges: solo los exchanges por defecto, todavía no existe transferencias.

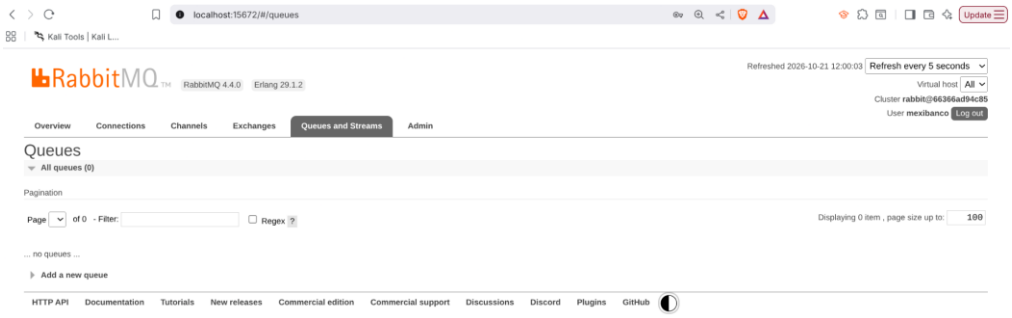


Ilustración 2. Pestaña Queues and Streams: ninguna cola (All queues 0).

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 5e06e04e16d0
Content-Type: application/json
Date: Wed, 21 Oct 2026 18:01:15 GMT

{"id":8,"clabe":"002180000000000001","titular":"Ana","saldo":1000}HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: acf38036aa7c
Content-Type: application/json
Date: Wed, 21 Oct 2026 18:01:14 GMT

{"id":9,"clabe":"002180000000000002","titular":"Beto","saldo":500}
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$
```

Ilustración 3. Alta de las cuentas de Ana (1000) y Beto (500): 201 Created en ambas.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ docker compose stop notificacion                # 1. apaga notificación

curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'   # 2. transfiere 50

curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'   # 3. ¿en qué estado quedó?
curl -s http://localhost:8080/api/cuentas/002180000000000001        #    saldo de Ana
curl -s http://localhost:8080/api/cuentas/002180000000000002        #    saldo de Beto

docker compose start notificacion               # 4. vuelve a encenderlo
[+] Stopping 1/1
 ✔ Container mexi-banco-v06-notificacion-1  Stopped                                                  3.3s
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Wed, 21 Oct 2026 18:02:36 GMT

{"timestamp":"2026-10-21T18:02:36.872+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
[{"id":9,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50.00,"fecha":"2026-10-21T1
8:02:36.483482Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actuali
zada":"2026-10-21T18:02:36.600323Z"}]{"id":8,"clabe":"002180000000000001","titular":"Ana","saldo":950.00}{"id":9,"c
labe[+] Running 4/4000002","titular":"Beto","saldo":550.00}
 ✔ Container mexi-banco-v06-db-1            Healthy                                                  0.5s
 ✔ Container mexi-banco-v06-discovery-1     Healthy                                                  0.5s
 ✔ Container mexi-banco-v06-rabbitmq-1      Healthy                                                  0.5s
 ✔ Container mexi-banco-v06-notificacion-1  Started                                                  0.3s
```

Ilustración 4. Parte 0: con notificación apagada la transferencia de 50 responde 500, queda en ABONADA y los saldos sí se movieron (Ana 950, Beto 550).

Momento	Saldo Ana	Saldo Beto
Antes de la transferencia	1000	500
Después (notificación apagada)	950	550

Qué respondió la transferencia. Un 500 Internal Server Error. Aun así, la consulta por estado ABONADA mostró la transferencia 9: existe y se quedó a medias, sin llegar a COMPLETADA. Los saldos sí cambiaron, así que el dinero salió de Ana y llegó a Beto.

Por qué pasó, en una frase. Después de mover el dinero, la saga llamaba por HTTP a notificación y esperaba su respuesta, como notificación estaba apagada, la excepción salió de ahí y la línea que marca COMPLETADA nunca se ejecutó, así que el cliente recibió un error aunque el cargo, el abono y los asientos ya habían ocurrido.

2. Publicar el evento [21/10/26 – 12:56 PM]

En vez de llamar por teléfono a notificación, ahora la transferencia deja un recado en un buzón (RabbitMQ) y sigue con lo suyo. Esto es lo que cambié en el servicio transferencia:

1. eventoId. Agregué a Transferencia un eventoId (UUID) que nace junto con la fila y se guarda con ella. Si el evento se publica dos veces lleva el mismo id, por eso lo uso después como llave de idempotencia.

2. El evento. Creé el record EventoTransferencia: es el contrato JSON con tipo (TransferenciaRealizada o TransferenciaRevertida), las CLABE, el monto y los saldos. Tiene un método routingKey() que decide entre transferencia.realizada y transferencia.revertida. Es un record propio y no la entidad para que cambiar una columna no rompa a quien escuche.

3. Exchange y JSON. ConfiguracionMensajeria declara el exchange transferencias, tipo topic y durable, y un convertidor para que los eventos viajen como JSON.

4. Publisher confirms. En application.yml activé la confirmación del broker y el rechazo de mensajes sin cola:

```
rabbitmq:
  connection-timeout: 2s
  publisher-confirm-type: correlated
  publisher-returns: true              # regresa el mensaje que ninguna cola recibió
  template:
    mandatory: true
```

5. El publicador. PublicadorDeEventos manda el evento con una CorrelationData y espera la confirmación. Lanza EventoNoPublicadoException si no hay conexión, si el broker responde nack, si no contesta a tiempo o si el mensaje regresa porque ninguna cola lo recibió. El mensaje es persistente (se escribe a disco).

6. La saga. En vez de las dos llamadas HTTP, primero guarda COMPLETADA y después publica. Si publicar falla no revierte nada: el dinero ya se movió bien, solo queda el error en el log.

```
t.marcarCompletada();
t = guardar(t);                       // primero la base dice COMPLETADA
avisar(EventoTransferencia.realizada(...)); // luego el recado

private void avisar(EventoTransferencia evento) {
    try { eventos.publicar(evento); }
    catch (RuntimeException sinPublicar) { log.error("... NO se publico: {}", ...); }
}
```

El orden importa: si fuera al revés, notificación podría avisar de una transferencia que la base todavía no dice que ocurrió.

Prueba con notificación apagada. Reconstruí transferencia y repetí la transferencia de 50. Ahora responde 201 y COMPLETADA (transferencia 10) y ya trae su eventoId. En el panel apareció el exchange transferencias (topic, con la marca D de durable), pero sin ningún binding, porque todavía nadie había declarado una cola. En el log, el publicador avisó que el evento no se publicó: ninguna cola lo recibió (NO_ROUTE). Esto pasa porque un exchange no guarda mensajes y, gracias a mandatory, el broker lo regresó en vez de tirarlo en silencio. Ese aviso se perdió, en la sección 3 notificación declara su cola.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ docker compose stop notificacion

curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'

docker compose logs transferencia | grep evento  # qué pasó con el evento
[+] Stopping 1/1
 ✔ Container mexi-banco-v06-notificacion-1  Stopped                                                    3.3s
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Wed, 21 Oct 2026 18:56:47 GMT

{"id":10,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50,"fecha":"2026-10-21T18:5
6:47.527504866Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,"intentosDeCompensacion":0,"actu
alizada":"2026-10-21T18:56:47.852085261Z","eventoId":"fc71d264-8f1a-46cb-ae02-4bab321df0d1"}transferencia-1  | 2026
-10-21T18:56:47.888Z ERROR 1 --- [transferencia] [nio-8080-exec-3] m.m.transferencia.TransferenciaService   : Trans
ferencia 10: quedo terminada pero el evento TransferenciaRealizada fc71d264-8f1a-46cb-ae02-4bab321df0d1 NO se publi
co: Ninguna cola recibio el evento fc71d264-8f1a-46cb-ae02-4bab321df0d1 con la routing key transferencia.realizada
(NO_ROUTE)

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ docker compose logs transferencia | grep evento
transferencia-1  | 2026-10-21T18:56:47.888Z ERROR 1 --- [transferencia] [nio-8080-exec-3] m.m.transferencia.Transfe
renciaService   : Transferencia 10: quedo terminada pero el evento TransferenciaRealizada fc71d264-8f1a-46cb-ae02-4
bab321df0d1 NO se publico: Ninguna cola recibio el evento fc71d264-8f1a-46cb-ae02-4bab321df0d1 con la routing key t
ransferencia.realizada (NO_ROUTE)
```

Ilustración 5. Transferencia 10 con notificación apagada: 201 COMPLETADA y el log con NO_ROUTE (ninguna cola recibió el evento).

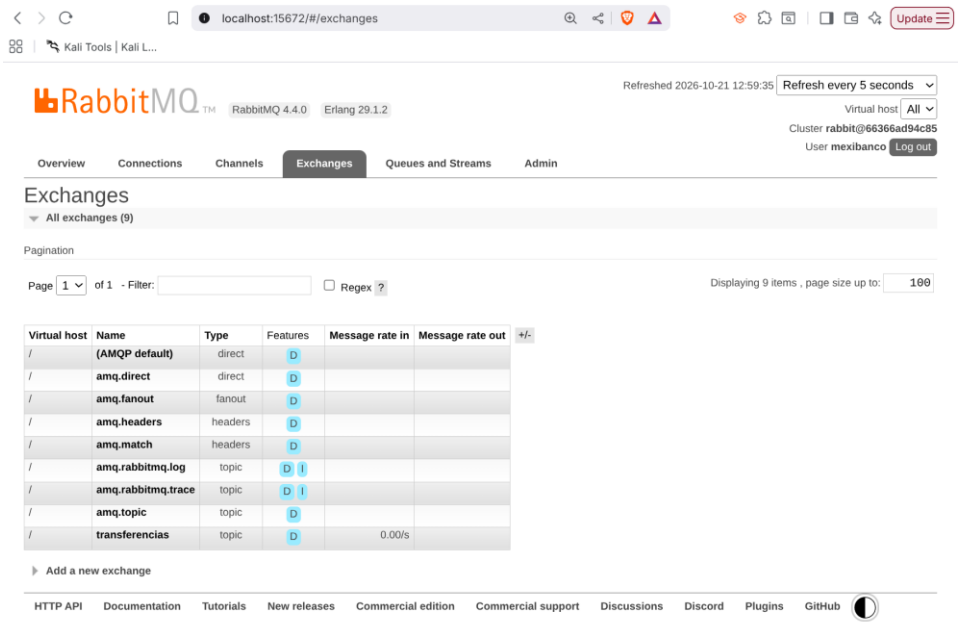


Ilustración 6. Pestaña Exchanges: ahora existe transferencias, topic y durable (D).

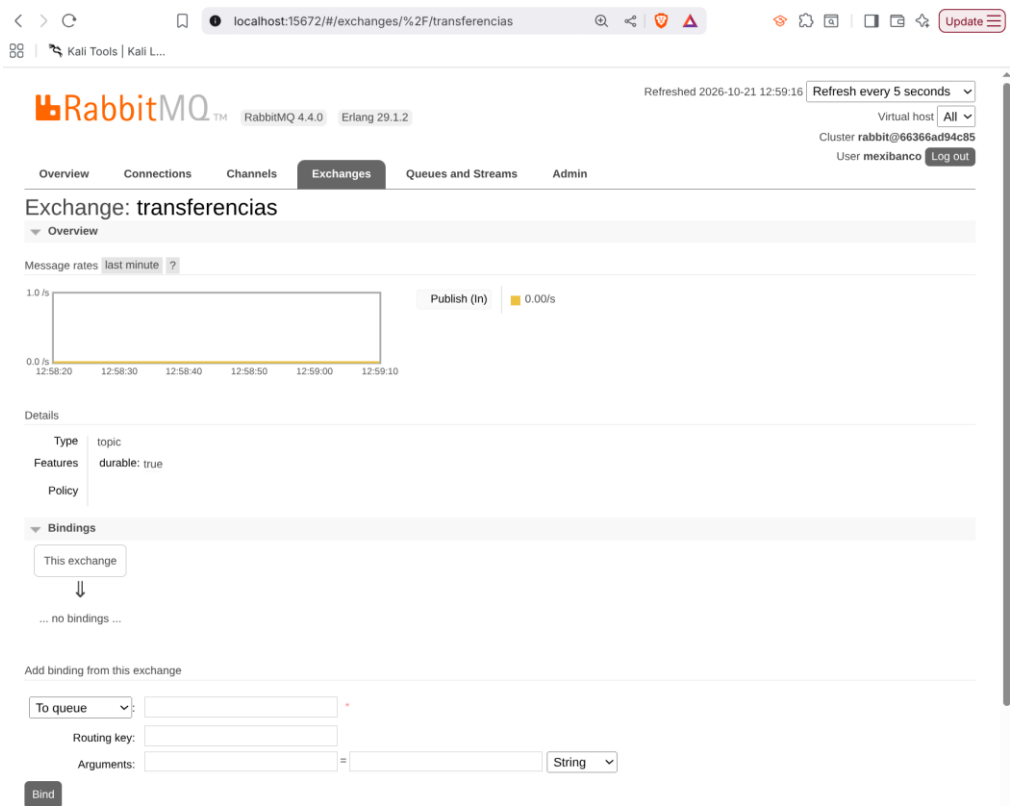


Ilustración 7. Detalle del exchange transferencias: tipo topic, durable true.

¿Qué garantiza el publisher confirm? Garantiza que el broker recibió el mensaje y lo aceptó (ack), o que lo rechazó (nack); junto con mandatory, además detecta si ninguna cola lo recibió, como vi con el NO_ROUTE. Lo que no garantiza es que alguien lo haya leído: con notificación apagada el broker confirma igual y el mensaje espera en la cola, porque la confirmación es entre el productor y el broker, no con el consumidor. Tampoco me protege si el proceso de transferencia muere justo entre guardar(t) y avisar(...): la base diría COMPLETADA y el evento nunca saldría. Y si el broker está caído no hay nada que confirmar.

3. Desacople en tiempo [21/10/26 – 1:23 PM]

Del lado de notificación construí tres piezas:

1. La cola. notificacion.transferencias, durable, enlazada al exchange con el patrón transferencia.*. Vive aunque notificación esté apagada, y ahí esperan los eventos.

2. Los avisos. avisarDeTransferencia guarda dos avisos por cada TransferenciaRealizada (a quien envió y a quien recibió) y uno por cada TransferenciaRevertida (a origen), todo en una sola transacción.

3. El consumidor con ack manual. Configuré acknowledge-mode: manual y prefetch: 5. El consumidor lee el JSON, guarda los avisos y solo entonces hace basicAck:

```
EventoTransferencia evento = json.readValue(mensaje.getBody(), EventoTransferencia.class);
List<Notificacion> avisos = notificaciones.avisarDeTransferencia(evento);  // commit
canal.basicAck(etiqueta, false);   // DESPUÉS de guardar
```

Prueba. Apagué notificación y mandé tres transferencias (11, 12 y 13): las tres respondieron 201. rabbitmqctl mostró la cola con 3 mensajes Ready y 0 Unacked. Al encender notificación, el primer curl a

la bandeja dio 503 porque todavía estaba arrancando; en el segundo ya aparecieron los avisos de 11, 12 y 13 en la bandeja de Beto. Este es el desacople en tiempo: productor y consumidor ya no tienen que estar vivos al mismo tiempo.



Ilustración 8. Tres transferencias (11, 12 y 13) con notificación apagada: 201 en las tres y la cola con 3 Ready.

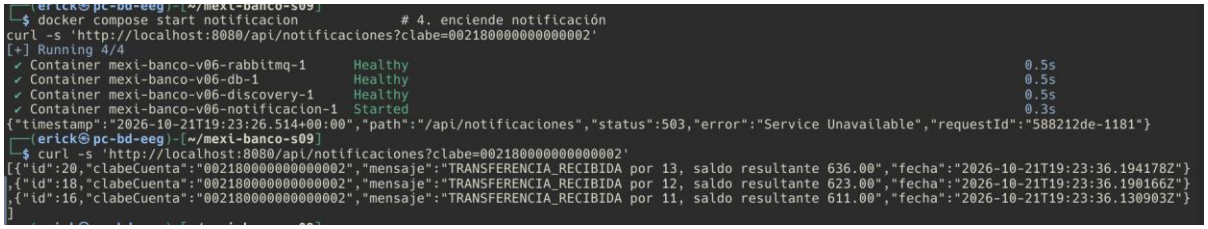


Ilustración 9. Al volver notificación: la bandeja de Beto tiene los avisos de 11, 12 y 13 (el primer 503 fue porque aún arrancaba).

El aviso de TransferenciaRevertida. Levanté transferencia con FALLAR_ABONO_A_CLABE apuntando a la cuenta de Beto y transferí 150 de Ana a Beto. El abono falló a propósito y la respuesta fue un 502 con la transferencia 14 en estado COMPENSACION_PENDIENTE: el dinero salió de origen y la devolución no pudo hacerse de inmediato. Después levanté transferencia sin la variable y el reintento programado de la S08 terminó la devolución: la transferencia quedó COMPENSADA (con 57 intentos de compensación), el saldo de Ana regresó a 864 y le llegó un aviso nuevo: DEVOLUCION_TRANSFERENCIA por 150. Hasta la S08 nadie le avisaba al cliente que su transferencia se había revertido.



Ilustración 10. Falla provocada del abono: 502 con la transferencia 14 en COMPENSACION_PENDIENTE.

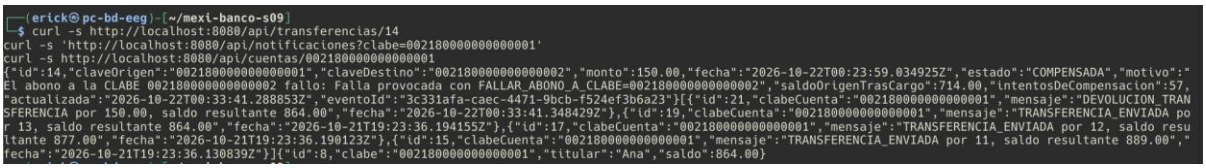


Ilustración 11. Transferencia 14 ya COMPENSADA, el aviso DEVOLUCION_TRANSFERENCIA por 150 y el saldo de Ana en 864.

¿Por qué el ack va después de guardar? Si lo pongo antes y el consumidor muere entre el ack y el guardado, el broker ya borró el mensaje y los avisos nunca se guardaron: el aviso se pierde (como máximo una vez). Si lo pongo después y muere entre el guardado y el ack, el broker no recibió el ack y vuelve a entregar el mensaje con redelivered=true, pero los avisos ya estaban guardados: el aviso se repite (al menos una vez). Elegí la segunda porque prefiero un aviso repetido a uno perdido, y el repetido se resuelve en la sección 4.

4. El duplicado [22/10/26 – 3:24 PM]

El "antes"

Cómo lo provoqué. Con el método del panel: apagué notificación, transferí 35 (transferencia 17, eventoId 98dd28ac…) y la cola quedó con 1 Ready. En Get messages (Nack message requeue true) copié el payload sin consumir el mensaje, y en Exchanges → transferencias lo publiqué otra vez con la routing key transferencia.realizada y el mismo eventoId. La cola quedó con 2 Ready. Después encendí notificación.

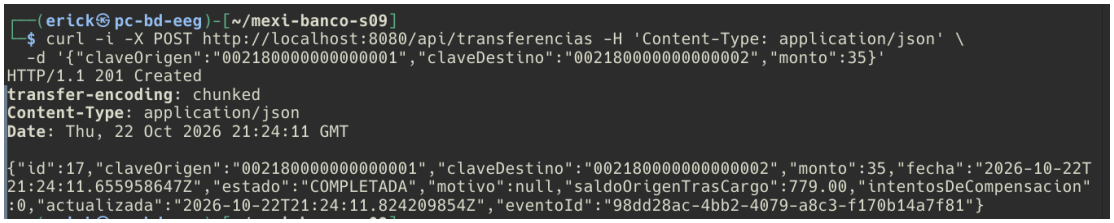


Ilustración 12. Transferencia 17 de 35 con notificación apagada: 201 COMPLETADA y su eventoId.

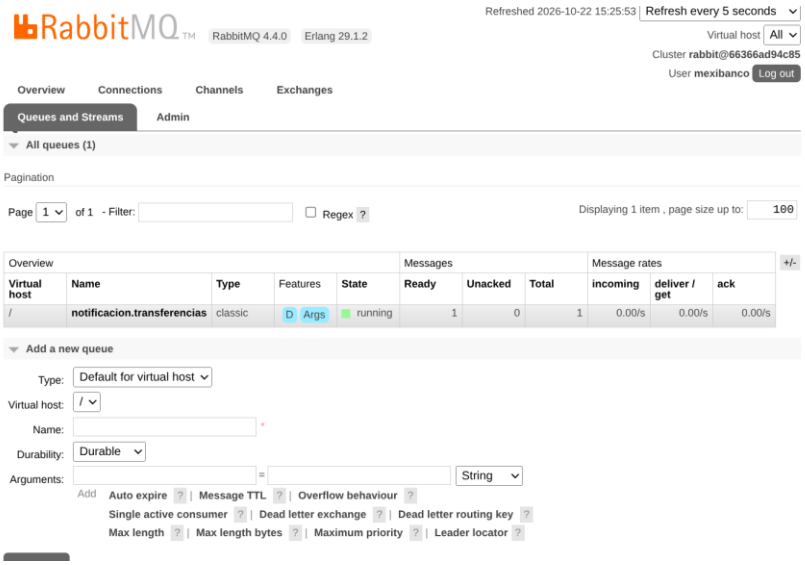


Ilustración 13. La cola tiene 1 Ready (el evento de la transferencia 17).

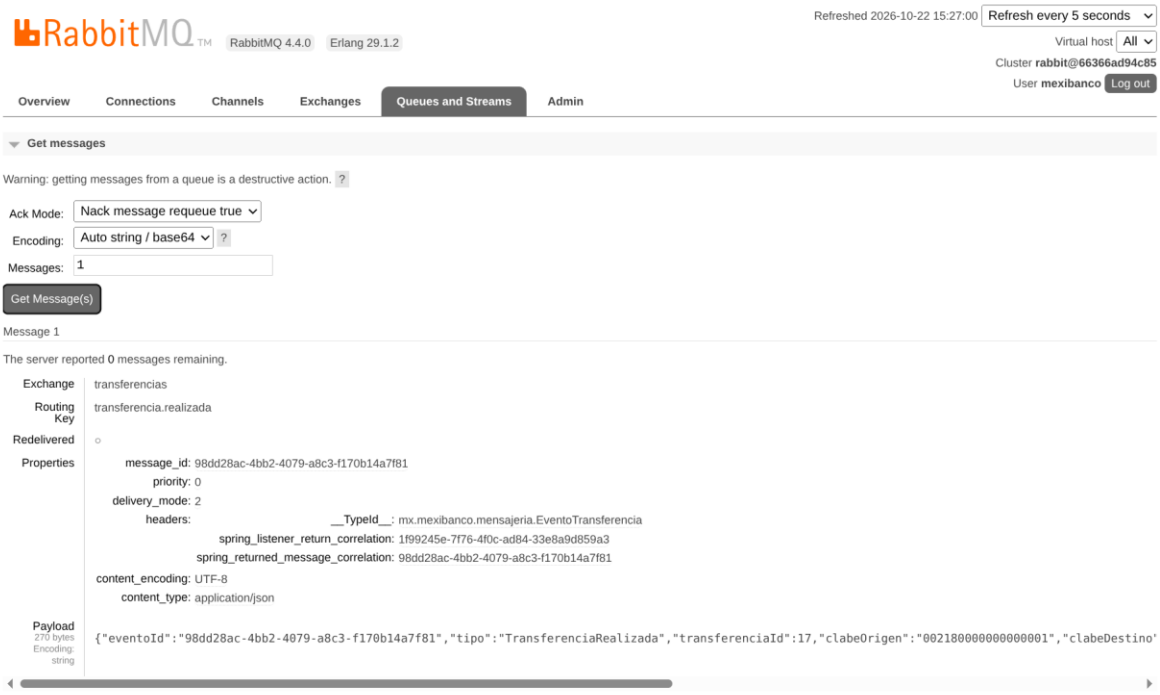


Ilustración 14. Get messages con Nack message requeue true: el mensaje se ve sin consumirse.

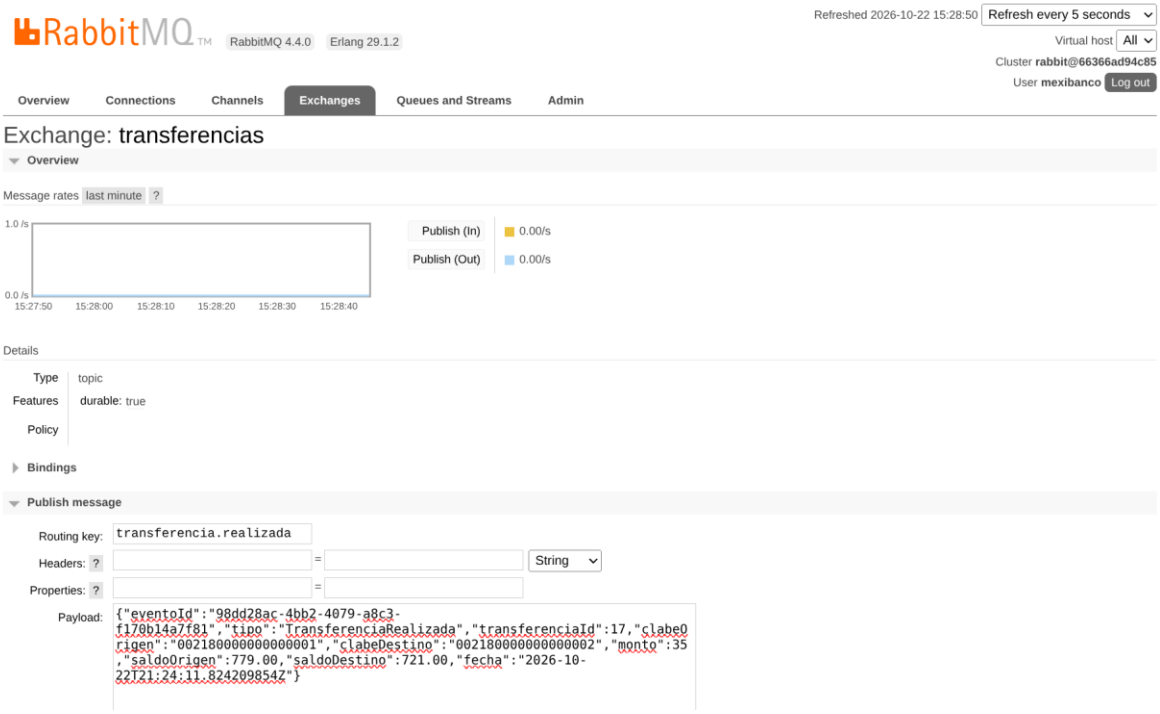


Ilustración 15. Publico el mismo payload (mismo eventoId) en el exchange transferencias.

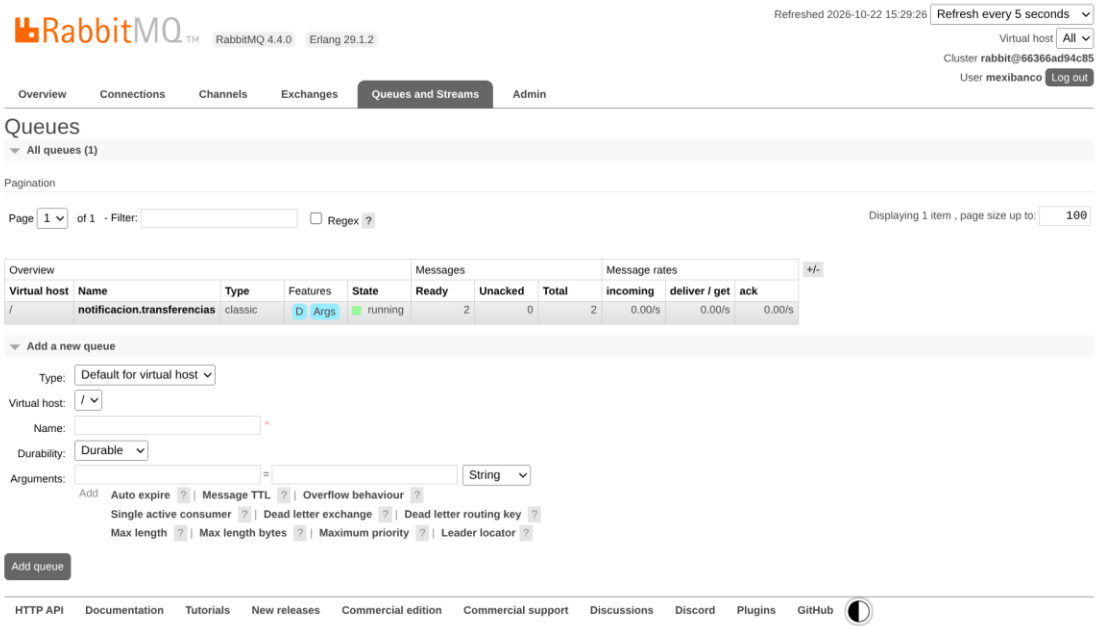


Ilustración 16. Ahora la cola tiene 2 Ready: el mismo evento dos veces.

Qué pasó. El mismo evento se procesó dos veces y cada entrega guardó su propio juego de avisos. En la bandeja de Beto quedaron los ids 29 y 31 (TRANSFERENCIA_RECIBIDA por 35, saldo 721.00) y en la de Ana los ids 28 y 30 (TRANSFERENCIA_ENVIADA por 35, saldo 779.00). En el log, las dos entregas del mismo evento dicen 2 aviso(s) nuevo(s) (la primera con redelivered=true, la espiada desde el panel, y la segunda con redelivered=false, la que publiqué yo).

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$  docker compose logs notificacion | grep -E "consumidor|ya procesado" | tail -n 4
notificacion-1  | 2026-10-22T21:30:54.280Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor]
TransferenciaRealizada de la transferencia 17 (evento 98dd28ac-4bb2-4079-a8c3-f170b14a7f81, redelivered=true): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-22T21:30:54.285Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor]
TransferenciaRealizada de la transferencia 17 (evento 98dd28ac-4bb2-4079-a8c3-f170b14a7f81, redelivered=false): 2 aviso(s) nuevo(s)
```

Ilustración 17. Log del consumidor: dos entregas del mismo evento, las dos con "2 aviso(s) nuevo(s)".

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":31,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 35, saldo resultante 721.00","fecha":"2026-10-22T21:30:54.282497Z"}
,{"id":29,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 35, saldo resultante 721.00","fecha":"2026-10-22T21:30:54.233093Z"}
,{"id":27,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 30, saldo resultante 686.00","fecha":"2026-10-22T21:19:19.310057Z"}
,{"id":25,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resultante 656.00","fecha":"2026-10-22T00:49:51.745681Z"}
,{"id":23,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resultante 656.00","fecha":"2026-10-22T00:49:51.669666Z"}
,{"id":20,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-21T19:23:36.194178Z"}
,{"id":18,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-21T19:23:36.190166Z"}
,{"id":16,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-21T19:23:36.130903Z"}
]
```

Ilustración 18. Bandeja de Beto: ids 29 y 31 idénticos (recibida por 35, saldo 721.00).

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":30,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 35, saldo resultante 779.00","fecha":"2026-10-22T21:30:54.282462Z"}
,{"id":28,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 35, saldo resultante 779.00","fecha":"2026-10-22T21:30:54.232998Z"}
,{"id":26,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 30, saldo resultante 814.00","fecha":"2026-10-22T21:19:19.309848Z"}
,{"id":24,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 20, saldo resultante 844.00","fecha":"2026-10-22T00:49:51.745649Z"}
,{"id":22,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 20, saldo resultante 844.00","fecha":"2026-10-22T00:49:51.669629Z"}
,{"id":21,"clabeCuenta":"002180000000000001","mensaje":"DEVOLUCION_TRANSFERENCIA por 150.00, saldo resultante 864.00","fecha":"2026-10-22T00:33:41.34
8429Z"},{"id":19,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00","fecha":"2026-10-21T19:23:36.19
4155Z"},{"id":17,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00","fecha":"2026-10-21T19:23:36.19
0123Z"},{"id":15,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00","fecha":"2026-10-21T19:23:36.13
0839Z"}]
```

Ilustración 19. Bandeja de Ana: ids 28 y 30 idénticos (enviada por 35, saldo 779.00).

Esto pasa porque RabbitMQ entrega al menos una vez y mi consumidor no se acordaba de que ya había procesado ese eventoId.

El arreglo

Hice el consumidor idempotente con una tabla de eventos ya procesados, en la misma transacción que los avisos:

1. Entidad EventoProcesado (tabla evento_procesado) con el eventoId como llave primaria, más tipo y transferenciaId.

2. En el repositorio, una sola sentencia atómica de Postgres que regresa 1 si el evento era nuevo y 0 si ya existía:

```
@Modifying
@Query(nativeQuery = true, value = """
  INSERT INTO evento_procesado (evento_id, tipo, transferencia_id, procesado_en)
  VALUES (:eventoId, :tipo, :transferenciaId, now())
  ON CONFLICT (evento_id) DO NOTHING""")
int marcarSiEsNuevo(UUID eventoId, String tipo, Long transferenciaId);
```

3. Al principio de avisarDeTransferencia, dentro del mismo @Transactional:

```
if (procesados.marcarSiEsNuevo(evento.eventoId(), evento.tipo(), evento.transferenciaId()) == 0)
{
    log.info("[notificacion] evento {} ya procesado, se ignora", evento.eventoId());
    return List.of();   // 0 avisos nuevos; el consumidor igual hace ack
}
```

Va en la misma transacción para que, si algo falla después de la marca, el rollback se lleve la marca y los avisos juntos y la siguiente entrega lo intente desde cero. Usé ON CONFLICT DO NOTHING y no "consultar y luego insertar" porque entre esas dos llamadas se podría colar otra entrega del mismo mensaje, y tampoco atrapo la violación de llave porque en Postgres deja la transacción abortada.

El "después"

Repetí el mismo experimento con otro monto: 40 (transferencia 19, eventoId 62827d9e…). Dejé la cola con 2 Ready y encendí notificación. La primera entrega guardó los 2 avisos; la segunda fue ignorada.

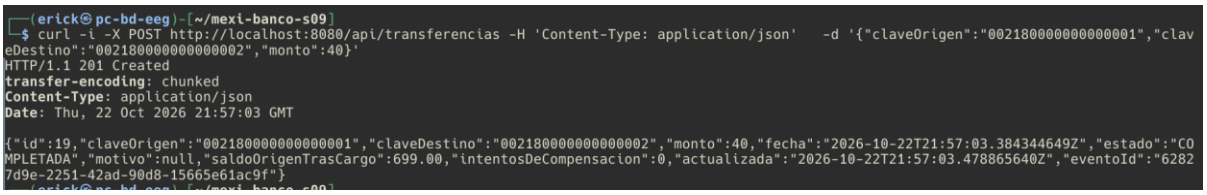


Ilustración 20. Transferencia 19 de 40 con notificación apagada: 201 COMPLETADA.

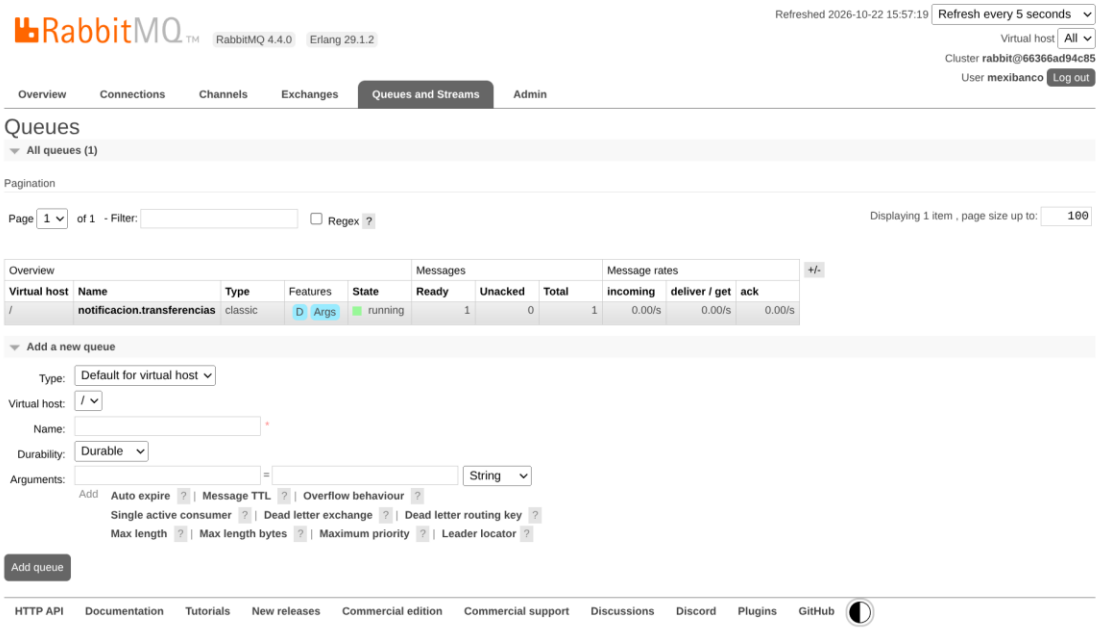


Ilustración 21. La cola con 1 Ready antes de duplicar.

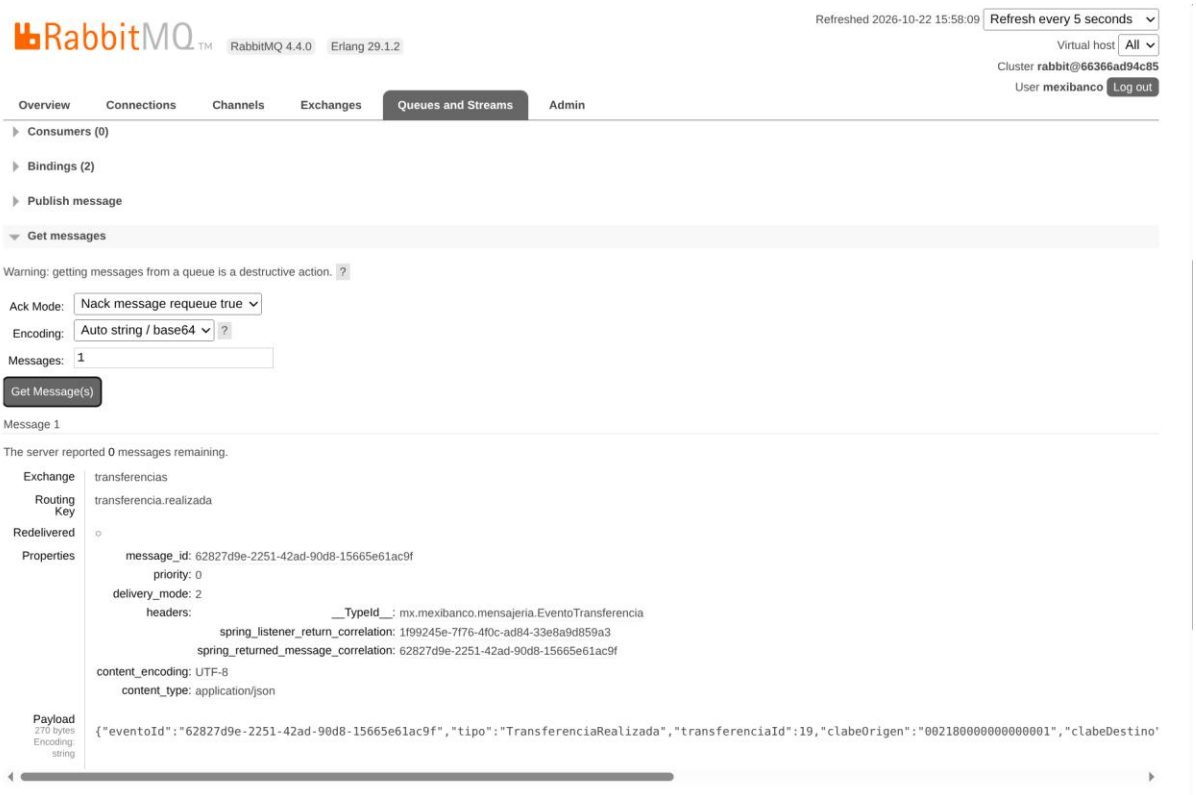


Ilustración 22. Get messages: el evento de la transferencia 19 (message_id 62827d9e…).

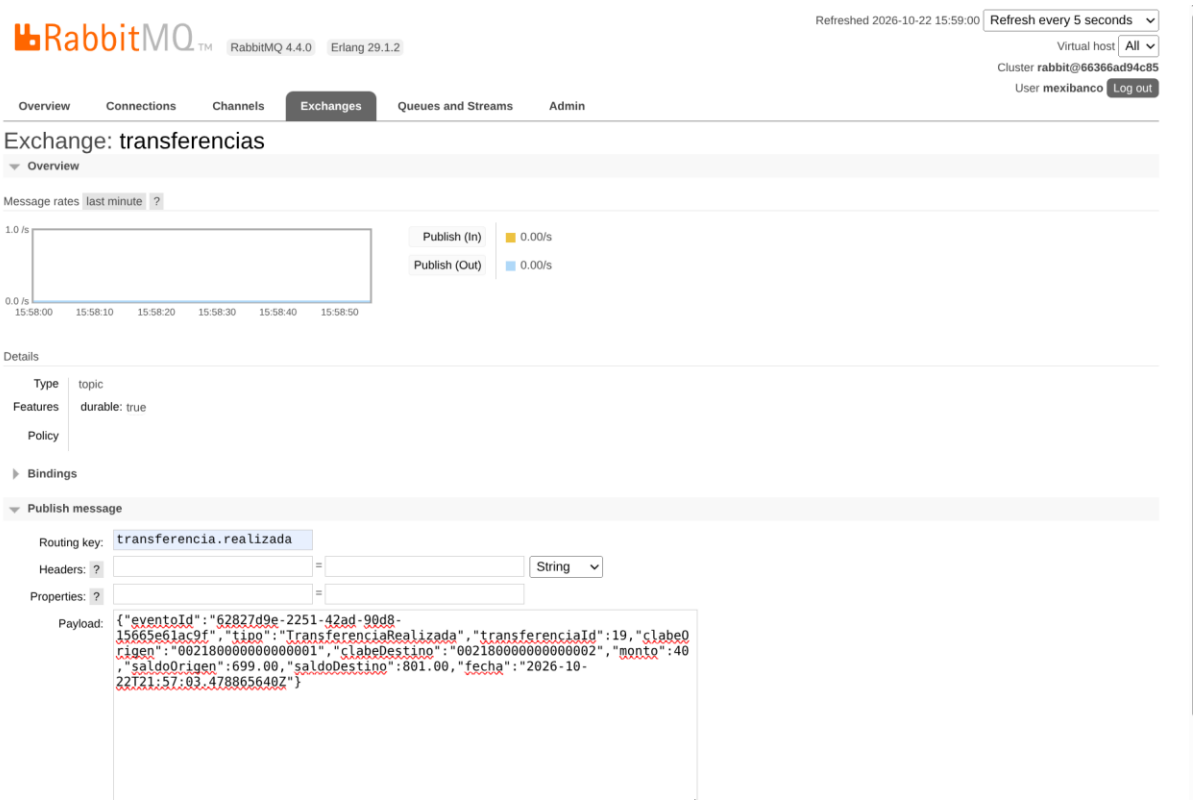


Ilustración 23. Publico el mismo payload otra vez.

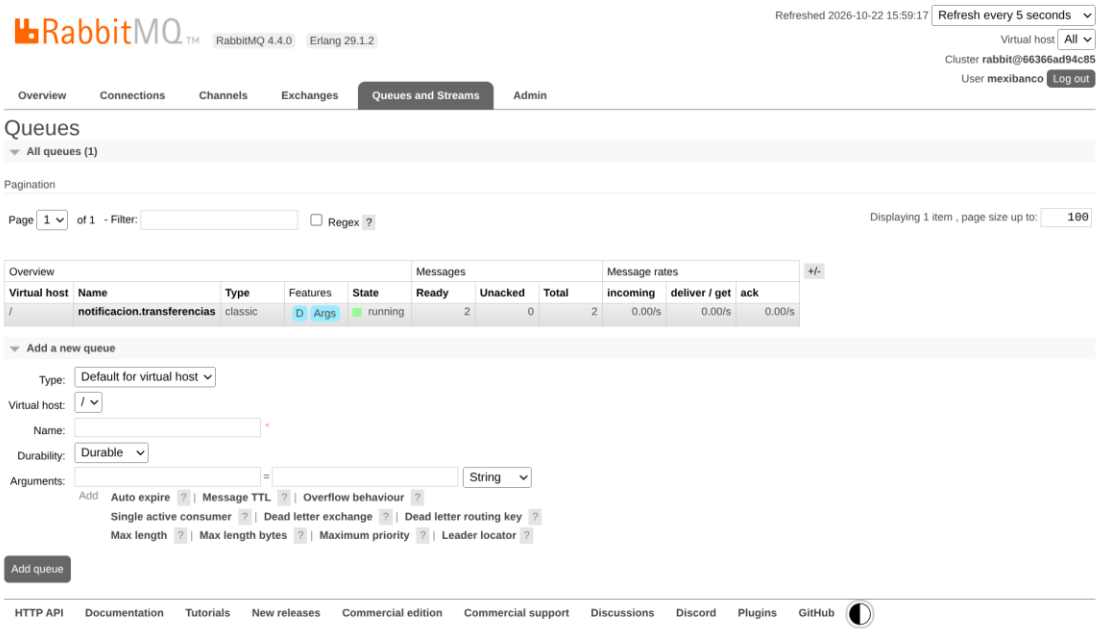


Ilustración 24. La cola con 2 Ready, igual que en el "antes".

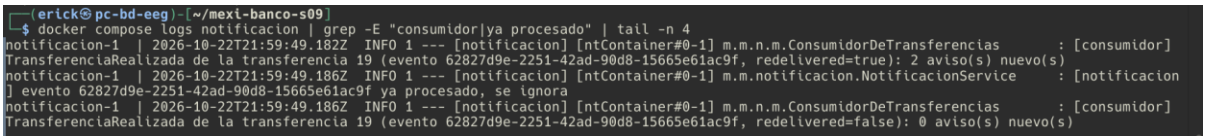


Ilustración 25. Log: primera entrega "2 aviso(s) nuevo(s)", después "ya procesado, se ignora" y la segunda entrega "0 aviso(s) nuevo(s)".

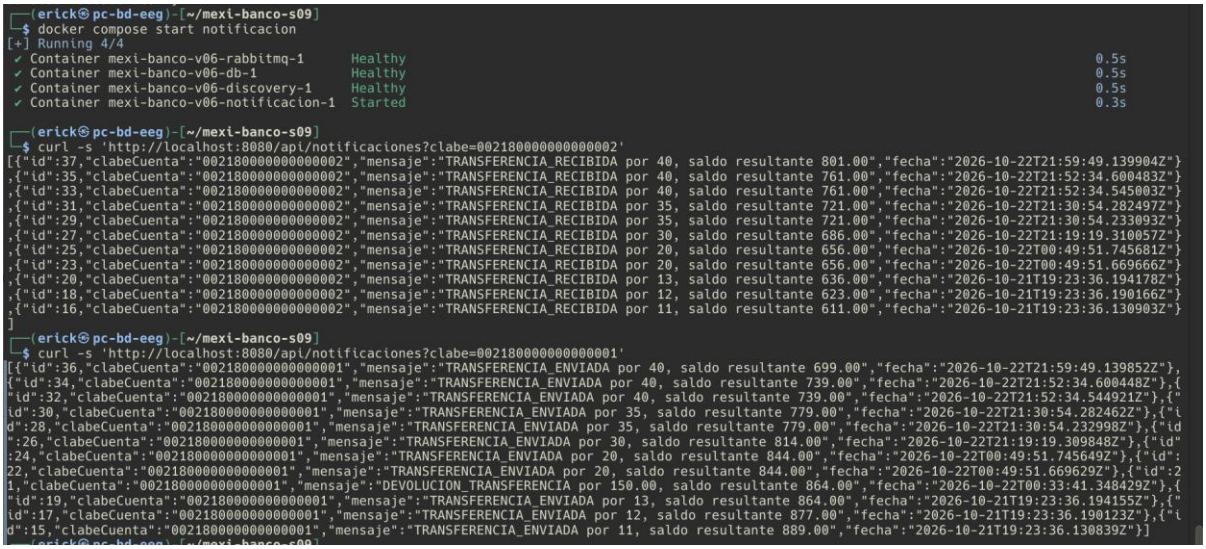


Ilustración 26. Bandejas después del arreglo: la transferencia de 40 aparece una sola vez en la de Beto (id 37, saldo 801.00) y en la de Ana (id 36, saldo 699.00).

Nota: los ids 32 a 35 (monto 40, saldo 739 y 761) vienen de un primer intento (transferencia 18) en el que sin querer probé sin reconstruir la imagen de notificación, por eso salieron duplicados y no tiene que ver con el arreglo final. La prueba válida es la de la transferencia 19.

5. La DLQ [22/10/26 – 4:07 PM]

El mensaje malo, antes del arreglo

Publiqué desde el panel, en el exchange transferencias con la routing key transferencia.realizada, un payload que no es un evento válido: {"hola": . Con el código de la sección 3, el consumidor no pudo leerlo: la cola quedó con 1 Unacked y el log mostró una JsonEOFException (Unexpected end-of-input) dentro de ConsumidorDeTransferencias.recibir. La excepción se escapa del método antes del basicAck: el mensaje no se confirma ni se rechaza, simplemente se queda sin procesar y sin que nadie lo revise. Una transferencia normal de 45 hecha después sí generó sus dos avisos, así que el consumidor seguía vivo.

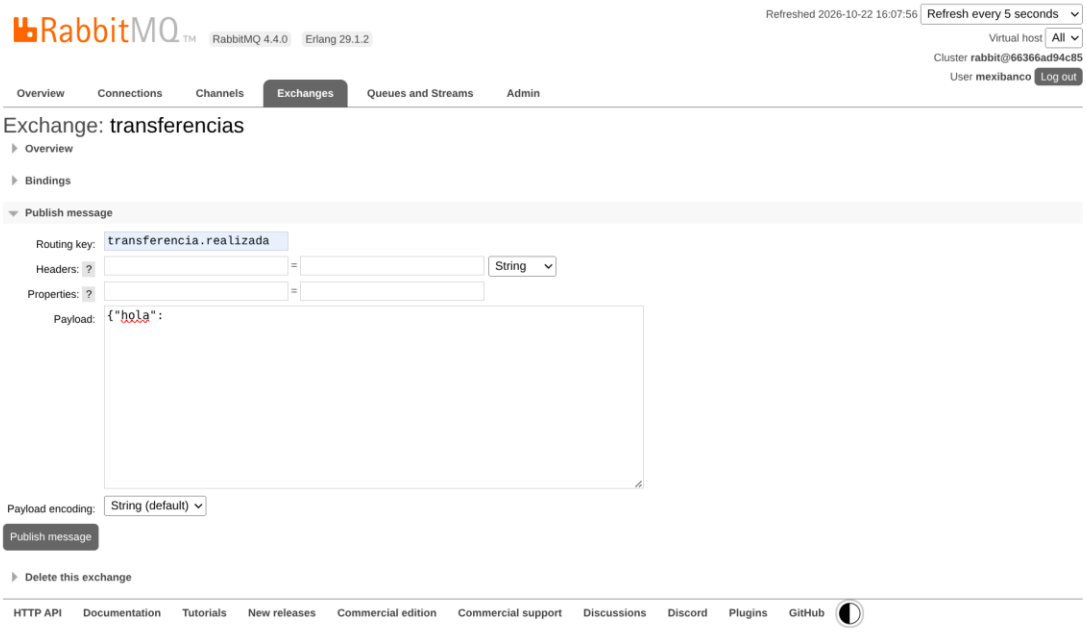


Ilustración 27. Publico el mensaje malo {"hola": con la routing key transferencia.realizada.

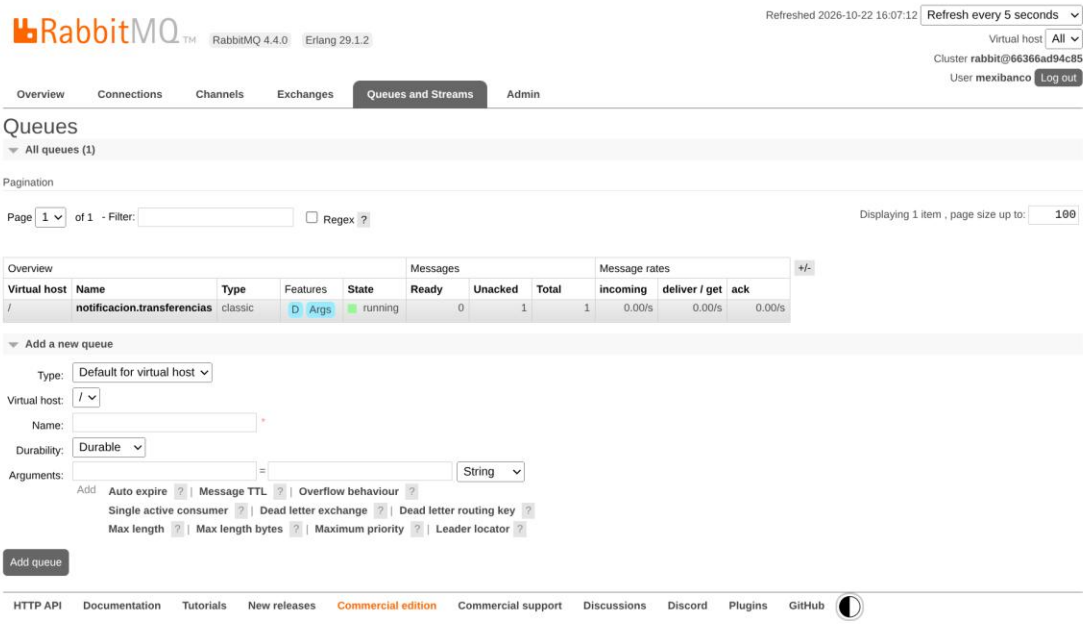


Ilustración 28. La cola queda con 0 Ready y 1 Unacked.

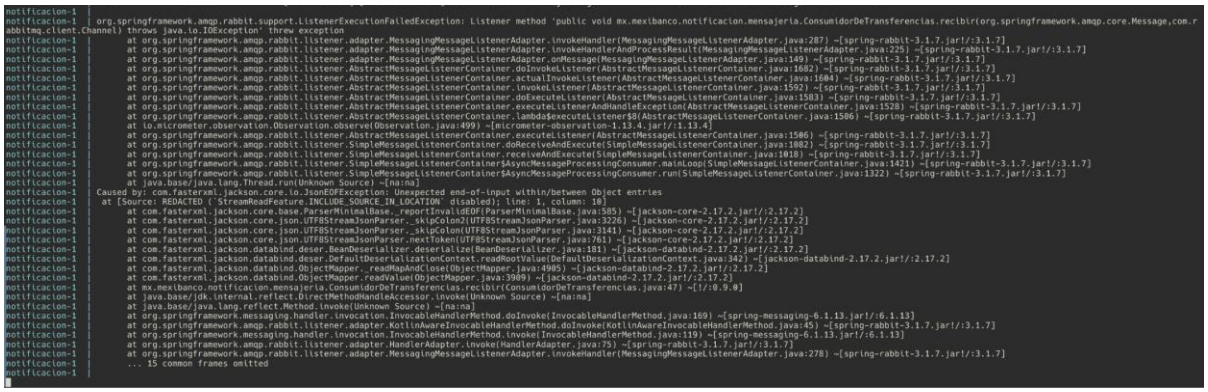


Ilustración 29. Log de notificación: JsonEOFException al leer el mensaje.

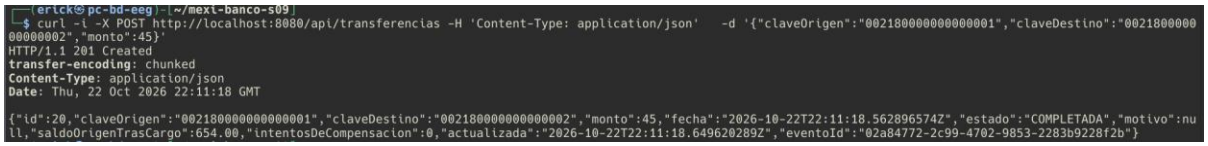


Ilustración 30. Una transferencia normal de 45 después del mensaje malo: 201 COMPLETADA.



Ilustración 31. Log: el consumidor procesó la transferencia 20 con normalidad (2 avisos).

El arreglo

1. Topología. Declaré la cola principal con los argumentos x-dead-letter-exchange (transferencias.dlx) y x-dead-letter-routing-key (notificacion.transferencias), el DLX transferencias.dlx (direct, durable) y la DLQ notificacion.transferencias.dlq (durable), enlazada con la clave notificacion.transferencias.

```
Queue cola = QueueBuilder.durable(COLA)
    .deadLetterExchange("transferencias.dlx")
    .deadLetterRoutingKey("notificacion.transferencias").build();
```

2. El consumidor ahora decide entre tres desenlaces. Si el JSON no se puede leer, o le falta el eventoId o el tipo, o el tipo no existe, el mensaje es veneno: reintentar no lo va a arreglar, así que hago basicNack sin reencolar y el broker lo manda a la DLQ. Si falla algo mío (por ejemplo la base de datos), el mensaje está bien, así que basicNack con reencolar para reintentar. Si todo sale bien, basicAck después de guardar.

```
canal.basicNack(etiqueta, false, false);  // veneno: requeue = false -> va a la DLQ
canal.basicNack(etiqueta, false, true);   // falla nuestra: requeue = true -> se reintenta
canal.basicAck(etiqueta, false);          // todo bien
```

3. PRECONDITION_FAILED. RabbitMQ no deja cambiar los argumentos de una cola que ya existe, así que borré notificacion.transferencias desde el panel (Delete Queue) y reconstruí notificación. Arrancó sin errores y declaró las dos colas.

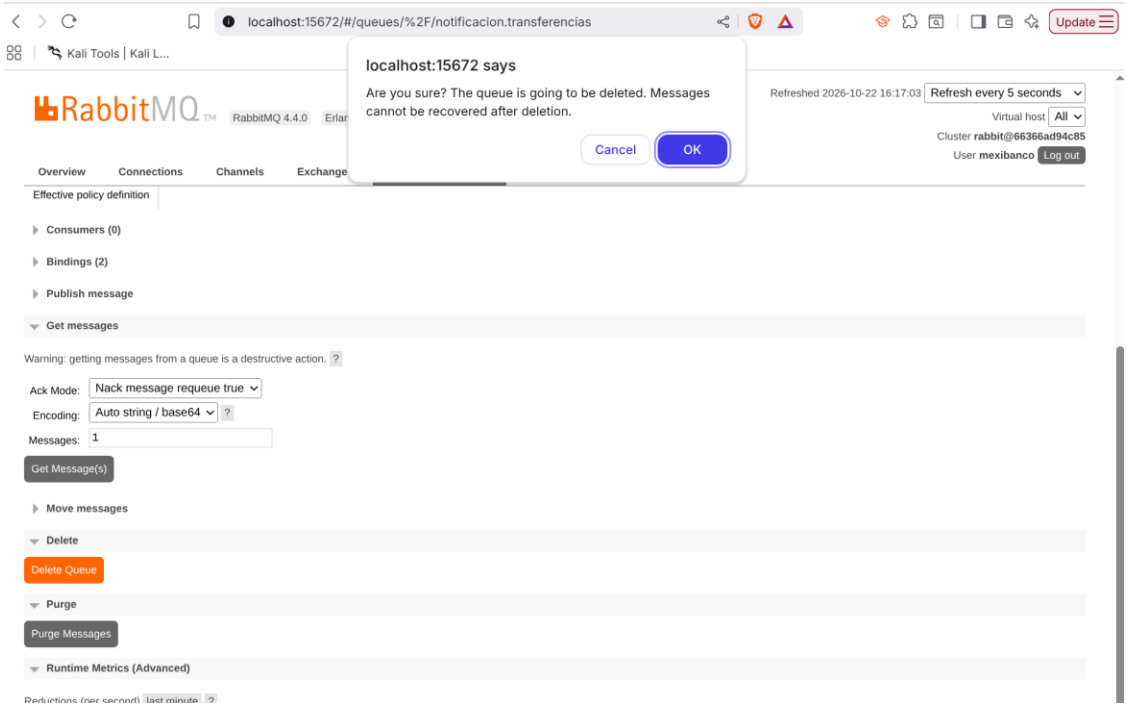


Ilustración 32. Borro la cola desde el panel para poder volver a declararla con los argumentos de la DLQ.

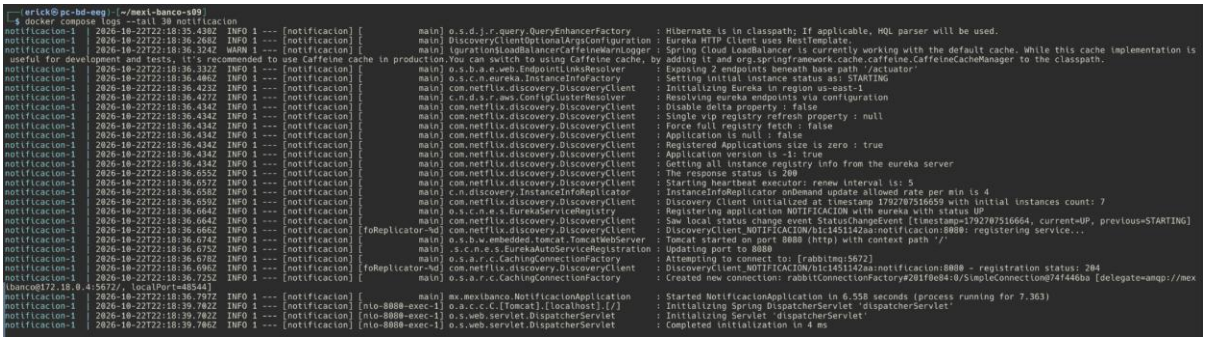


Ilustración 33. Notificación reconstruida: arranca sin PRECONDITION_FAILED.

El "después"

Volví a publicar el mismo mensaje malo. En el log aparece "mensaje ilegible, va a la DLQ", la cola principal queda en 0 Ready y 0 Unacked, y notificacion.transferencias.dlq tiene 1 Ready. Al ver el mensaje en la DLQ, la cabecera x-death indica reason: rejected, con la cola de origen notificacion.transferencias y la routing key transferencia.realizada. Una transferencia normal de 50 (transferencia 21) siguió generando sus avisos: el id 41 en la bandeja de Beto.



Ilustración 34. Publico otra vez el mensaje malo.

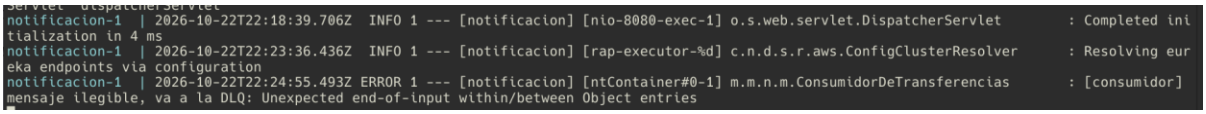


Ilustración 35. Log: "mensaje ilegible, va a la DLQ", sin la pila de excepción.

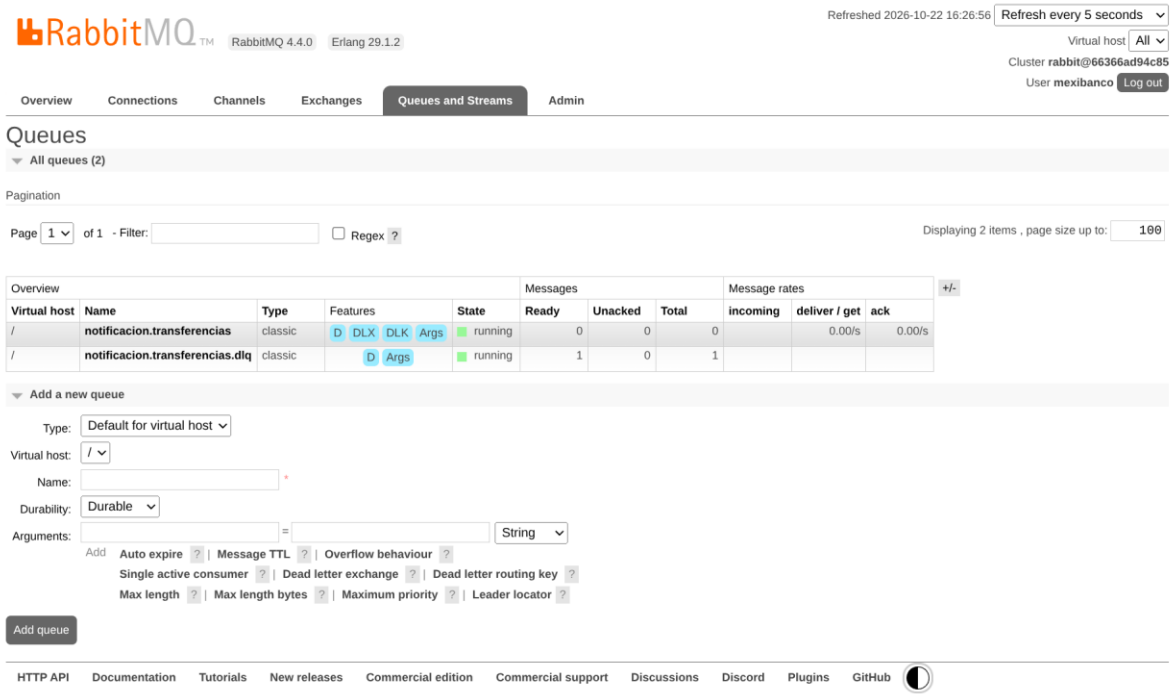


Ilustración 36. Las dos colas: la principal (con DLX y DLK) en 0 y la DLQ con 1 Ready.

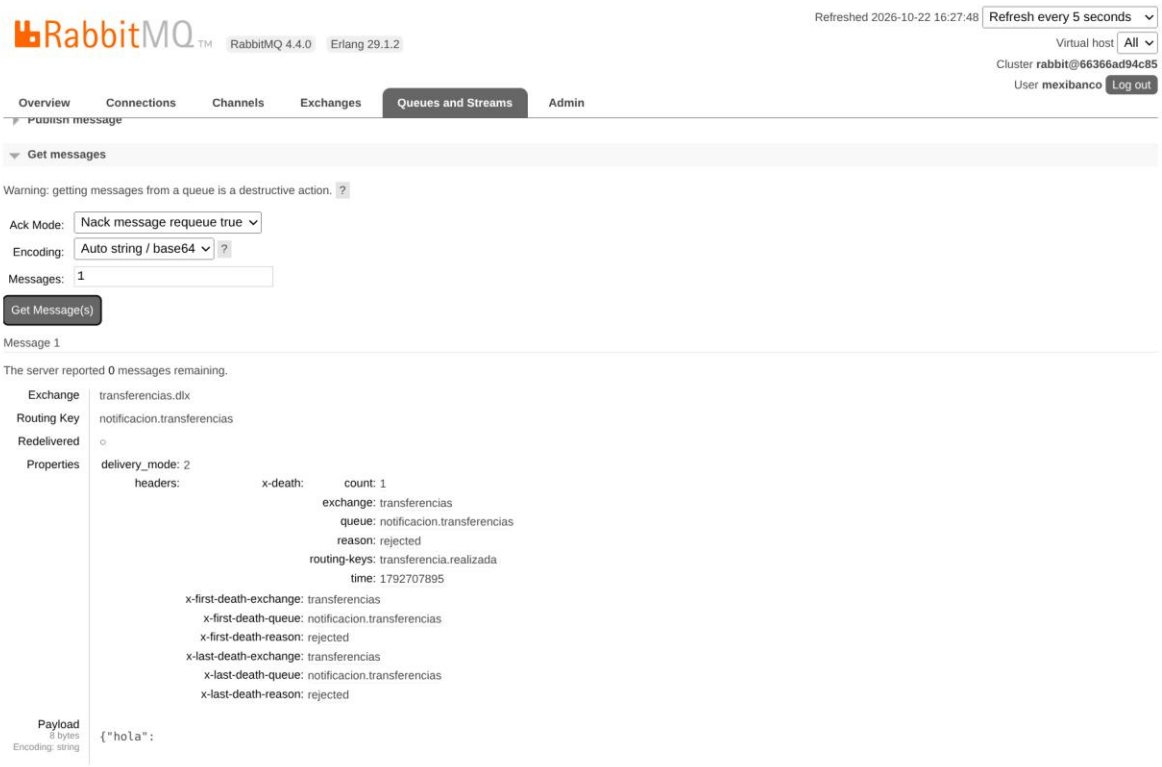


Ilustración 37. El mensaje en la DLQ: x-death con reason rejected y el payload {"hola": .

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Thu, 22 Oct 2026 22:29:07 GMT

{"id":21,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50,"fecha":"2026-10-22T22:29:07.140464379Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":604.00,"intentosDeCompensacion":0,"actualizada":"2026-10-22T22:29:07.232366012Z","eventoId":"cd332752-1e1d-4e82-93d2-10fb094eb48d"}
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ curl -s 'http://localhost:8080/api/notificaciones?clabe=00218000000000002'
[{"id":41,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 50, saldo resultante 896.00","fecha":"2026-10-22T22:29:07.374896Z"}
,{"id":39,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 45, saldo resultante 846.00","fecha":"2026-10-22T22:11:18.664764Z"}
,{"id":37,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 40, saldo resultante 801.00","fecha":"2026-10-22T21:59:49.139904Z"}
,{"id":35,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 40, saldo resultante 761.00","fecha":"2026-10-22T21:52:34.600483Z"}
,{"id":33,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 40, saldo resultante 761.00","fecha":"2026-10-22T21:52:34.545003Z"}
,{"id":31,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 35, saldo resultante 721.00","fecha":"2026-10-22T21:30:54.282497Z"}
,{"id":29,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 35, saldo resultante 721.00","fecha":"2026-10-22T21:30:54.233093Z"}
,{"id":27,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 30, saldo resultante 686.00","fecha":"2026-10-22T21:19:19.310057Z"}
,{"id":25,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resultante 656.00","fecha":"2026-10-22T00:49:51.745681Z"}
,{"id":23,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resultante 656.00","fecha":"2026-10-22T00:49:51.669666Z"}
,{"id":20,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-21T19:23:36.194178Z"}
,{"id":18,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-21T19:23:36.190166Z"}
,{"id":16,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-21T19:23:36.130903Z"}
]
```

Ilustración 38. Transferencia de 50 después del arreglo: 201 y su aviso (id 41) en la bandeja de Beto.

6. Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso? Con mi versión de las secciones 2 a 5, la transferencia termina bien y responde 201, porque avisar() atrapa la excepción de publicar y solo la escribe en el log; pero el aviso se pierde para siempre: la base dice COMPLETADA y el evento nunca sale, y cuando RabbitMQ vuelve nadie lo manda. El publisher confirm no lo arregla porque solo me dice si el broker recibió el mensaje; si el broker está caído no hay conexión ni nadie que confirme, y si transferencia muere entre guardar y publicar tampoco hay nada que confirmar. Es la doble escritura: la base y el broker son dos sistemas sin una transacción que los una. El Outbox de la sección 7 lo resuelve.

2. ¿Por qué el ack va después de guardar? Porque así nunca pierdo un aviso. Si el ack va antes, un fallo entre el ack y el guardado borra el mensaje sin que se hayan guardado los avisos (garantía de como máximo una vez: no hay duplicados, pero hay pérdidas). Si va después, un fallo entre el guardado y el ack hace que el broker reentregue (garantía de al menos una vez: no hay pérdidas, pero puede haber duplicados). Escogí la segunda y compensé el duplicado con la idempotencia por eventoId.

3. Elección y sacrificio. Elección: publicar un evento en RabbitMQ en vez de llamar por HTTP a notificación. Porque: así el aviso ya no tumba una transferencia que ya movió el dinero, y notificación puede estar apagada y ponerse al día después. Sacrifico: la consistencia inmediata (el cliente puede ver su saldo cambiado antes de que le llegue el aviso), la certeza de que el aviso llegó (el 201 ya no me dice si se avisó), y la simplicidad, porque ahora debo manejar duplicados (idempotencia), mensajes venenos (DLQ) y la doble escritura (Outbox).

7. El Outbox (opcional) [22/10/26 – 8:08 PM]

La solución fue un Transactional Outbox en el servicio transferencia:

1. Tabla outbox. La entidad EventoSaliente guarda el eventoId (único), la routing key, el evento serializado a JSON, cuándo se creó y cuándo se publicó (publicado_en, vacío mientras está pendiente).

2. Escribir en la misma transacción. OutboxDeEventos.guardarYEncolar guarda la transferencia en su estado final y la fila del outbox en una sola transacción local, con un TransactionTemplate (un @Transactional llamado desde la misma clase no haría nada). La saga ya no habla con RabbitMQ.

```
Transferencia fila = transferencias.save(t);
outbox.save(new EventoSaliente(evento.eventoId(), evento.routingKey(), payload));
```

3. El relay. RelayDeOutbox es un @Scheduled que cada segundo toma las filas pendientes, las publica con PublicadorDeEventos (que espera la confirmación) y les pone publicado_en. Si el broker no confirma, corta el lote y la fila queda pendiente para la siguiente vuelta. La consulta usa FOR UPDATE SKIP LOCKED para que, con varias réplicas, cada relay tome filas distintas.

Prueba. Apagué RabbitMQ y transferí 1: respondió 201 COMPLETADA (transferencia 22). La tabla outbox tenía la fila con publicado_en vacío y el log del relay decía que el evento seguía pendiente (UnknownHostException: rabbitmq, porque el contenedor estaba detenido). Con el broker apagado, además, el health de transferencia respondió 503 y el contenedor pasó a unhealthy. Al encender RabbitMQ, el relay publicó el evento solo: el log dice "1 evento(s) publicado(s) y confirmado(s) por el broker", publicado_en quedó con fecha (02:17:39 UTC) y 110 milisegundos después llegó el aviso TRANSFERENCIA_RECIBIDA por 1 a la bandeja de Beto. El health volvió a UP.

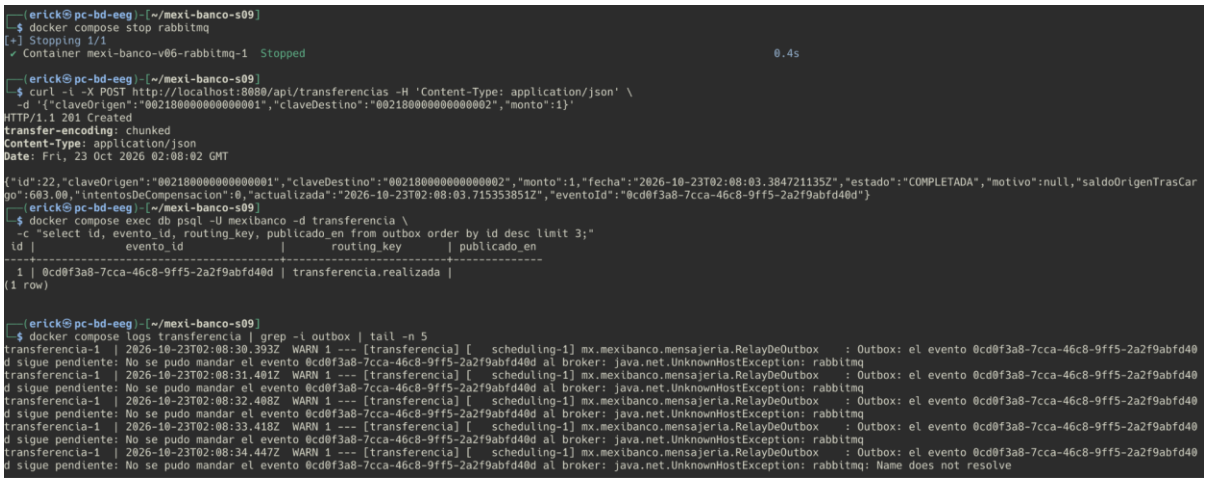


Ilustración 39. Con RabbitMQ apagado: 201 COMPLETADA, la fila del outbox con publicado_en vacío y el relay reintentando (sigue pendiente).

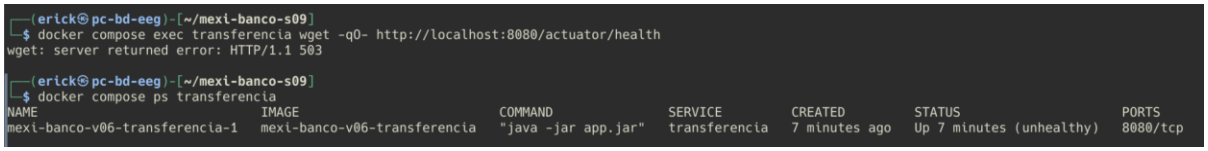


Ilustración 40. Con el broker apagado, el health de transferencia responde 503 y el contenedor está unhealthy.



Ilustración 41. RabbitMQ encendido: el relay publica y confirma el evento y publicado_en queda con fecha.

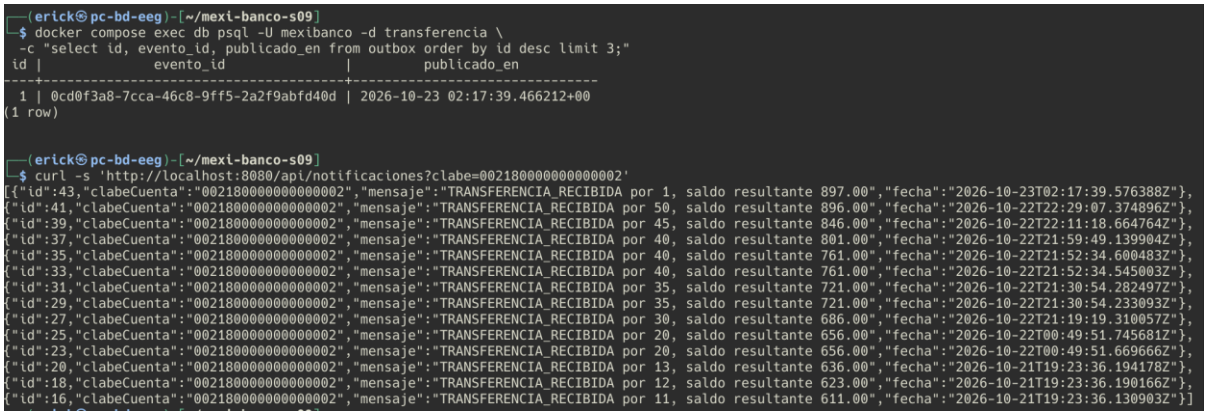


Ilustración 42. La fila del outbox ya publicada y el aviso TRANSFERENCIA_RECIBIDA por 1 (id 43) en la bandeja de Beto.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s09]
└─$ docker compose exec transferencia wget -qO- http://localhost:8080/actuator/health
{"status":"UP","groups":["liveness","readiness"]}
```

Ilustración 43. El health de transferencia vuelve a UP.

¿Quién me salva si el relay publica y muere antes de marcar la fila? El evento saldría otra vez con el mismo eventoId, y lo absorbe el consumidor idempotente de la sección 4: la segunda entrega responde 0 aviso(s) nuevo(s). El outbox garantiza al menos una vez y la idempotencia convierte eso en efecto una sola vez.

¿Transferencia sigue healthy con el broker apagado? No: vi el 503 y el estado unhealthy, porque /actuator/health de Spring Boot incluye el indicador de RabbitMQ. La transferencia funciona sin el broker, pero el contenedor se marca como enfermo; se podría arreglar excluyendo ese indicador del health.